

Práctica 3: Analizador sintáctico de descenso predictivo

Compiladores 2026-2

Profesora: Ariel Adara Mercado Martínez

Alumno

Edwin Rodríguez Nevarez - 312227441

Objetivo

Que el alumno conozca e implemente los principios del análisis sintáctico descendente predictivo, aplicando la construcción de los conjuntos FIRST y FOLLOW, así como la tabla de análisis sintáctico predictivo, para construir un analizador sintáctico LL(1) que, junto con el analizador léxico desarrollado con Flex, permita reconocer programas válidos del lenguaje C_1.

Desarrollo

1. Determinar los conjuntos N , Σ y el símbolo inicial S .

A partir de las reglas de producción de la gramática original $G = (N, \Sigma, P, S)$, definimos:

■ Conjunto de No-Terminales (N):

$N = \{\text{programa, declaraciones, declaracion, tipo, lista_var, sentencias, sentencia, expresion}\}$

■ Conjunto de Terminales (Σ):

$\Sigma = \{\text{int, float, if, else, while, identificador, numero, ;, ,, =, +, -, *, /, (,)}\}$

■ Símbolo inicial (S): $S = \text{programa}$

a) Proceso de eliminación de ambigüedad.

La gramática original es ambigua en las producciones de *expresion*, porque no tiene precedencia ni asociatividad para los operadores aritméticos. Para eliminarla, hicimos los No-Terminales *termino* y *factor* para hacer la jerarquía estándar:

- $\text{expresion} \rightarrow \text{expresion} + \text{termino} \mid \text{expresion} - \text{termino} \mid \text{termino}$
- $\text{termino} \rightarrow \text{termino} * \text{factor} \mid \text{termino} / \text{factor} \mid \text{factor}$
- $\text{factor} \rightarrow (\text{expresion}) \mid \text{identificador} \mid \text{numero}$

b) Proceso de eliminación de la recursividad izquierda.

Se tiene recursividad por la izquierda en *declaraciones*, *lista_var*, *sentencias*, *expresion* y *termino*. Se aplicó la transformación $A \rightarrow A\alpha \mid \beta \implies A \rightarrow \beta A', A' \rightarrow \alpha A' \mid \epsilon$:

- $\text{declaraciones} \rightarrow \text{declaracion declaraciones}'$

- $declaraciones' \rightarrow declaracion\ declaraciones' \mid \epsilon$
- $lista_var \rightarrow \text{identificador}\ lista_var'$
- $lista_var' \rightarrow ,\ \text{identificador}\ lista_var' \mid \epsilon$
- $sentencias \rightarrow sentencia\ sentencias'$
- $sentencias' \rightarrow sentencia\ sentencias' \mid \epsilon$
- $expresion \rightarrow termino\ expresion'$
- $expresion' \rightarrow +\ termino\ expresion' \mid -\ termino\ expresion' \mid \epsilon$
- $termino \rightarrow factor\ termino'$
- $termino' \rightarrow *\ factor\ termino' \mid /\ factor\ termino' \mid \epsilon$

c) **Proceso de factorización izquierda.**

No es necesario aplicar factorización izquierda. Después de eliminar la ambigüedad y la recursividad izquierda, ninguna regla de un mismo No-Terminal comparte un prefijo.

d) **Mostrar los nuevos conjuntos N' y P' , que definen G' .**

Nuevo conjunto de No-Terminales (N'):

$$N' = \{\text{programa}, \text{declaraciones}, \text{declaraciones}', \text{declaracion}, \text{tipo}, \text{lista_var}, \text{lista_var}', \text{sentencias}, \text{sentencias}', \text{sentencia}, \text{expresion}, \text{expresion}', \text{termino}, \text{termino}', \text{factor}\}$$

Nuevo conjunto de Producciones (P'):

- $programa \rightarrow declaraciones\ sentencias$
- $declaraciones \rightarrow declaracion\ declaraciones'$
- $declaraciones' \rightarrow declaracion\ declaraciones' \mid \epsilon$
- $declaracion \rightarrow tipo\ lista_var\ ;$
- $tipo \rightarrow \text{int} \mid \text{float}$
- $lista_var \rightarrow \text{identificador}\ lista_var'$
- $lista_var' \rightarrow ,\ \text{identificador}\ lista_var' \mid \epsilon$
- $sentencias \rightarrow sentencia\ sentencias'$
- $sentencias' \rightarrow sentencia\ sentencias' \mid \epsilon$
- $sentencia \rightarrow \text{identificador} = expresion\ ; \mid \text{if} (expresion)sentencias\ \text{else}\ sentencias \mid \text{while} (expresion)sentencias$
- $expresion \rightarrow termino\ expresion'$
- $expresion' \rightarrow +\ termino\ expresion' \mid -\ termino\ expresion' \mid \epsilon$
- $termino \rightarrow factor\ termino'$
- $termino' \rightarrow *\ factor\ termino' \mid /\ factor\ termino' \mid \epsilon$
- $factor \rightarrow (expresion) \mid \text{identificador} \mid \text{numero}$

2. **Mostrar en el archivo la construcción de los conjuntos FIRST de la gramática G' .**

El conjunto $FIRST(A)$ está formado por los símbolos terminales que pueden aparecer en la primera posición de cualquier cadena derivada a partir del No-Terminal A . Si A puede derivar en la cadena vacía, se incluye ϵ :

- $FIRST(\text{tipo}) = \{\mathbf{int}, \mathbf{float}\}$
- $FIRST(\text{lista_var}') = \{., \epsilon\}$
- $FIRST(\text{lista_var}) = \{\mathbf{identificador}\}$
- $FIRST(\text{declaracion}) = FIRST(\text{tipo}) = \{\mathbf{int}, \mathbf{float}\}$
- $FIRST(\text{declaraciones}') = FIRST(\text{declaracion}) \cup \{\epsilon\} = \{\mathbf{int}, \mathbf{float}, \epsilon\}$
- $FIRST(\text{declaraciones}) = FIRST(\text{declaracion}) = \{\mathbf{int}, \mathbf{float}\}$
- $FIRST(\text{factor}) = \{(\mathbf{, identificador, numero}\}$
- $FIRST(\text{termino}') = \{*, /, \epsilon\}$
- $FIRST(\text{termino}) = FIRST(\text{factor}) = \{(\mathbf{, identificador, numero}\}$
- $FIRST(\text{expresion}') = \{+, -, \epsilon\}$
- $FIRST(\text{expresion}) = FIRST(\text{termino}) = \{(\mathbf{, identificador, numero}\}$
- $FIRST(\text{sentencia}) = \{\mathbf{identificador, if, while}\}$
- $FIRST(\text{sentencias}') = FIRST(\text{sentencia}) \cup \{\epsilon\} = \{\mathbf{identificador, if, while, \epsilon}\}$
- $FIRST(\text{sentencias}) = FIRST(\text{sentencia}) = \{\mathbf{identificador, if, while}\}$
- $FIRST(\text{programa}) = FIRST(\text{declaraciones}) = \{\mathbf{int, float}\}$

3. Mostrar en el archivo la construcción de los conjuntos FOLLOW de la gramática G' .

Para calcular el conjunto $FOLLOW(A)$ aplicamos tres reglas:

- a) Si A es el símbolo inicial, se añade el símbolo de fin de cadena ($\$$).
- b) Si existe una producción $B \rightarrow \alpha A \beta$, todo lo que esté en $FIRST(\beta)$ (excepto ϵ) se añade a $FOLLOW(A)$.
- c) Si existe $B \rightarrow \alpha A$, o si $B \rightarrow \alpha A \beta$ y $FIRST(\beta)$ contiene ϵ , entonces todo $FOLLOW(B)$ se añade a $FOLLOW(A)$.

Aplicando estas reglas paso a paso a los No-Terminales de G' :

- **programa:** Es el símbolo inicial (Regla 1).
 $\Rightarrow FOLLOW(\text{programa}) = \{\$\}$
- **declaraciones:** Aparece en $\text{programa} \rightarrow \text{declaraciones sentencias}$. Le sigue sentencias , por lo que aplicamos la Regla 2.
 $\Rightarrow FOLLOW(\text{declaraciones}) = FIRST(\text{sentencias}) = \{\mathbf{identificador, if, while}\}$
- **declaraciones':** Aparece al final de $\text{declaraciones} \rightarrow \text{declaracion declaraciones'}$. Por la Regla 3, hereda de declaraciones .
 $\Rightarrow FOLLOW(\text{declaraciones}') = FOLLOW(\text{declaraciones}) = \{\mathbf{identificador, if, while}\}$

- **declaracion:** Aparece en $declaraciones \rightarrow declaracion\ declaraciones'$. Le sigue $declaraciones'$ (Regla 2), aportando $\{\mathbf{int}, \mathbf{float}\}$. Como $declaraciones'$ deriva en ϵ , hereda de $declaraciones$ aportando $\{\mathbf{identificador}, \mathbf{if}, \mathbf{while}\}$ (Regla 3).
 $\Rightarrow FOLLOW(declaracion) = \{\mathbf{int}, \mathbf{float}, \mathbf{identificador}, \mathbf{if}, \mathbf{while}\}$
- **tipo:** Aparece en $declaracion \rightarrow tipo\ lista_var\ ;$. Le sigue $lista_var$ (Regla 2).
 $\Rightarrow FOLLOW(tipo) = FIRST(lista_var) = \{\mathbf{identificador}\}$
- **lista_var:** Aparece en $declaracion \rightarrow tipo\ lista_var\ ;$. Le sigue el terminal $;$ (Regla 2).
 $\Rightarrow FOLLOW(lista_var) = \{;\}$
- **lista_var':** Aparece al final de $lista_var \rightarrow \mathbf{identificador}\ lista_var'$. Hereda de $lista_var$ (Regla 3).
 $\Rightarrow FOLLOW(lista_var') = FOLLOW(lista_var) = \{;\}$
- **sentencias:** Aparece en tres lugares: 1) $programa \rightarrow declaraciones\ sentencias$ (hereda $\$$ por Regla 3). 2) $sentencia \rightarrow \mathbf{if}\ (\dots)\ sentencias\ \mathbf{else}\ \dots$ (le sigue $\mathbf{else}$ por Regla 2). 3) $sentencia \rightarrow \mathbf{while}\ (\dots)\ sentencias$ (hereda el FOLLOW de $sentencia$ por Regla 3).
 $\Rightarrow FOLLOW(sentencias) = \{\mathbf{else}, \$\} \cup FOLLOW(sentencia) = \{\mathbf{identificador}, \mathbf{if}, \mathbf{while}, \mathbf{else}, \$\}$
- **sentencias':** Aparece al final de $sentencias \rightarrow sentencia\ sentencias'$. Hereda de $sentencias$ (Regla 3).
 $\Rightarrow FOLLOW(sentencias') = FOLLOW(sentencias) = \{\mathbf{identificador}, \mathbf{if}, \mathbf{while}, \mathbf{else}, \$\}$
- **sentencia:** Aparece en $sentencias \rightarrow sentencia\ sentencias'$. Le sigue $sentencias'$ aportando $\{\mathbf{identificador}, \mathbf{if}, \mathbf{while}\}$. Como deriva en ϵ , hereda de $sentencias$ aportando $\{\mathbf{else}, \$\}$.
 $\Rightarrow FOLLOW(sentencia) = \{\mathbf{identificador}, \mathbf{if}, \mathbf{while}, \mathbf{else}, \$\}$
- **expresion:** Aparece seguido de $;$ en asignaciones y seguido de $)$ en ciclos, condicionales y factores (Regla 2).
 $\Rightarrow FOLLOW(expresion) = \{;,)\}$
- **expresion':** Aparece al final de $expresion \rightarrow termino\ expresion'$. Hereda de $expresion$ (Regla 3).
 $\Rightarrow FOLLOW(expresion') = FOLLOW(expresion) = \{;,)\}$
- **termino:** Aparece en $expresion \rightarrow termino\ expresion'$. Le sigue $expresion'$ aportando $\{+, -\}$ (Regla 2). Como deriva en ϵ , hereda de $expresion$ aportando $\{;,)\}$ (Regla 3).
 $\Rightarrow FOLLOW(termino) = \{+, -, ;,)\}$
- **termino':** Aparece al final de $termino \rightarrow factor\ termino'$. Hereda de $termino$ (Regla 3).
 $\Rightarrow FOLLOW(termino') = FOLLOW(termino) = \{+, -, ;,)\}$
- **factor:** Aparece en $termino \rightarrow factor\ termino'$. Le sigue $termino'$ aportando $\{*, /\}$. Como deriva en ϵ , hereda de $termino$ aportando $\{+, -, ;,)\}$.
 $\Rightarrow FOLLOW(factor) = \{*, /, +, -, ;,)\}$

4. Mostrar en el archivo la construcción de la tabla de análisis sintáctico predictivo para G' .

La tabla de análisis sintáctico predictivo M se construye con los conjuntos FIRST y FOLLOW. Por razones de espacio, en lugar de dibujar una tabla de 15×17 , se hizo una lista con las celdas válidas $M[A, a]$, donde A es un No-Terminal y a es un Terminal. Las celdas que no están son errores de sintaxis.

Obtenemos las siguientes entradas válidas para la tabla M :

Para el No-Terminal *programa*:

- $M[\text{programa}, \text{int}] = \text{programa} \rightarrow \text{declaraciones sentencias}$
- $M[\text{programa}, \text{float}] = \text{programa} \rightarrow \text{declaraciones sentencias}$

Para el No-Terminal *declaraciones*:

- $M[\text{declaraciones}, \text{int}] = \text{declaraciones} \rightarrow \text{declaracion declaraciones'}$
- $M[\text{declaraciones}, \text{float}] = \text{declaraciones} \rightarrow \text{declaracion declaraciones'}$

Para el No-Terminal *declaraciones'*:

- $M[\text{declaraciones'}, \text{int}] = \text{declaraciones'} \rightarrow \text{declaracion declaraciones'}$
- $M[\text{declaraciones'}, \text{float}] = \text{declaraciones'} \rightarrow \text{declaracion declaraciones'}$
- $M[\text{declaraciones'}, \text{identificador}] = \text{declaraciones'} \rightarrow \epsilon$
- $M[\text{declaraciones'}, \text{if}] = \text{declaraciones'} \rightarrow \epsilon$
- $M[\text{declaraciones'}, \text{while}] = \text{declaraciones'} \rightarrow \epsilon$

Para el No-Terminal *declaracion*:

- $M[\text{declaracion}, \text{int}] = \text{declaracion} \rightarrow \text{tipo lista_var ;}$
- $M[\text{declaracion}, \text{float}] = \text{declaracion} \rightarrow \text{tipo lista_var ;}$

Para el No-Terminal *tipo*:

- $M[\text{tipo}, \text{int}] = \text{tipo} \rightarrow \text{int}$
- $M[\text{tipo}, \text{float}] = \text{tipo} \rightarrow \text{float}$

Para el No-Terminal *lista_var*:

- $M[\text{lista_var}, \text{identificador}] = \text{lista_var} \rightarrow \text{identificador lista_var'}$

Para el No-Terminal *lista_var'*:

- $M[\text{lista_var'}, ,] = \text{lista_var'} \rightarrow , \text{identificador lista_var'}$
- $M[\text{lista_var'}, ;] = \text{lista_var'} \rightarrow \epsilon$

Para el No-Terminal *sentencias*:

- $M[\text{sentencias}, \text{identificador}] = \text{sentencias} \rightarrow \text{sentencia sentencias'}$
- $M[\text{sentencias}, \text{if}] = \text{sentencias} \rightarrow \text{sentencia sentencias'}$

- $M[\text{sentencias}, \mathbf{while}] = \text{sentencias} \rightarrow \text{sentencia sentencias}'$

Para el No-Terminal *sentencias'*:

- $M[\text{sentencias}', \mathbf{identificador}] = \text{sentencias}' \rightarrow \text{sentencia sentencias}'$
- $M[\text{sentencias}', \mathbf{if}] = \text{sentencias}' \rightarrow \text{sentencia sentencias}'$
- $M[\text{sentencias}', \mathbf{while}] = \text{sentencias}' \rightarrow \text{sentencia sentencias}'$
- $M[\text{sentencias}', \mathbf{else}] = \text{sentencias}' \rightarrow \epsilon$
- $M[\text{sentencias}', \$] = \text{sentencias}' \rightarrow \epsilon$

Para el No-Terminal *sentencia*:

- $M[\text{sentencia}, \mathbf{identificador}] = \text{sentencia} \rightarrow \mathbf{identificador} = \text{expresion} ;$
- $M[\text{sentencia}, \mathbf{if}] = \text{sentencia} \rightarrow \mathbf{if} (\text{expresion}) \text{sentencias} \mathbf{else} \text{sentencias}$
- $M[\text{sentencia}, \mathbf{while}] = \text{sentencia} \rightarrow \mathbf{while} (\text{expresion}) \text{sentencias}$

Para el No-Terminal *expresion*:

- $M[\text{expresion}, \mathbf{identificador}] = \text{expresion} \rightarrow \text{termino expresion}'$
- $M[\text{expresion}, \mathbf{numero}] = \text{expresion} \rightarrow \text{termino expresion}'$
- $M[\text{expresion}, (] = \text{expresion} \rightarrow \text{termino expresion}'$

Para el No-Terminal *expresion'*:

- $M[\text{expresion}', +] = \text{expresion}' \rightarrow + \text{termino expresion}'$
- $M[\text{expresion}', -] = \text{expresion}' \rightarrow - \text{termino expresion}'$
- $M[\text{expresion}', ;] = \text{expresion}' \rightarrow \epsilon$
- $M[\text{expresion}',)] = \text{expresion}' \rightarrow \epsilon$

Para el No-Terminal *termino*:

- $M[\text{termino}, \mathbf{identificador}] = \text{termino} \rightarrow \text{factor termino}'$
- $M[\text{termino}, \mathbf{numero}] = \text{termino} \rightarrow \text{factor termino}'$
- $M[\text{termino}, (] = \text{termino} \rightarrow \text{factor termino}'$

Para el No-Terminal *termino'*:

- $M[\text{termino}', *] = \text{termino}' \rightarrow * \text{factor termino}'$
- $M[\text{termino}', /] = \text{termino}' \rightarrow / \text{factor termino}'$
- $M[\text{termino}', +] = \text{termino}' \rightarrow \epsilon$
- $M[\text{termino}', -] = \text{termino}' \rightarrow \epsilon$
- $M[\text{termino}', ;] = \text{termino}' \rightarrow \epsilon$
- $M[\text{termino}',)] = \text{termino}' \rightarrow \epsilon$

Para el No-Terminal *factor*:

- $M[\text{factor}, \text{identificador}] = \text{factor} \rightarrow \text{identificador}$
- $M[\text{factor}, \text{numero}] = \text{factor} \rightarrow \text{numero}$
- $M[\text{factor}, ()] = \text{factor} \rightarrow (\text{expresion})$

5. Sustituir el contenido del Analizador Léxico (*lexer.ll*) con el implementado en la segunda práctica.

```
1  %{
2  #include <iostream>
3  #include <string>
4  using namespace std;
5  #include "headers/Symbols.hpp"
6  #include "headers/Lexer.hpp"
7  int line=1;
8  %}
9
10 %option c++
11 %option outfile="Lexer.cpp"
12 %option yyclass="Lexer"
13
14 ID      [a-zA-Z_][a-zA-Z0-9_]*
15 NUMERO  [0-9]+(\.[0-9]+)?([eE][+-]?[0-9]+)?
16
17 %%
18 [ \t\r]+ { /* Ignoramos espacios */ }
19 "\n"     { line++; }
20 "int"     { return INT; }
21 "float"   { return FLOAT; }
22 "if"      { return IF; }
23 "else"    { return ELSE; }
24 "while"   { return WHILE; }
25 ";"       { return PYC; }
26 ","       { return COMA; }
27 "="       { return ASIG; }
28 "+"       { return MAS; }
29 "-"       { return MENOS; }
30 "*"       { return MUL; }
31 "/"       { return DIV; }
32 "("       { return LPAR; }
33 ")"       { return RPAR; }
34 {ID}      { return IDENTIFICADOR; }
35 {NUMERO}  { return NUMERO; }
36 .         { cout << "ERROR LEXICO en linea " << line << ": " << yytext
37             << endl; }
38 %%
39
40 int yyFlexLexer::yywrap(){ return 1; }
41 int Lexer::getLine() { return line; }
```

6. Definir en un comentario de *Symbols.hpp* la gramática G' .

7. Definir Σ en un enum de *Symbols.hpp*.
8. Definir N' en un enum de *Symbols.hpp*.

A continuación se muestra el código de headers/Symbols.hpp:

```

1 #ifndef __SYMBOLS_HPP__
2 #define __SYMBOLS_HPP__
3
4 /** * 6. Definir en un comentario la gramatica G'(N', T, P', S)
5  * S = programa
6  * N' = {programa, declaraciones, declaraciones_prima, declaracion,
7  * tipo,
8  * lista_var, lista_var_prima, sentencias, sentencias_prima, sentencia
9  * expresion, expresion_prima, termino, termino_prima, factor}
10 * T = {int, float, if, else, while, identificador, numero, ;, ,, =,
11 +, -, *, /, (, ), $}
12 * P' = {
13 * programa -> declaraciones sentencias
14 * declaraciones -> declaracion declaraciones_prima
15 * declaraciones_prima -> declaracion declaraciones_prima | epsilon
16 * declaracion -> tipo lista_var ;
17 * tipo -> int | float
18 * lista_var -> identificador lista_var_prima
19 * lista_var_prima -> , identificador lista_var_prima | epsilon
20 * sentencias -> sentencia sentencias_prima
21 * sentencias_prima -> sentencia sentencias_prima | epsilon
22 * sentencia -> identificador = expresion ; | if ( expresion )
23 sentencias else sentencias | while ( expresion ) sentencias
24 * expresion -> termino expresion_prima
25 * expresion_prima -> + termino expresion_prima | - termino
26 expresion_prima | epsilon
27 * termino -> factor termino_prima
28 * termino_prima -> * factor termino_prima | / factor termino_prima |
29 epsilon
30 * factor -> ( expresion ) | identificador | numero
31 * }
32 */
33
34 enum Token {
35     t_eof = 0, primero,
36     INT, FLOAT, IF, ELSE, WHILE,
37     PYC, COMA, ASIG,
38     MAS, MENOS, MUL, DIV,
39     LPAR, RPAR,
40     IDENTIFICADOR, NUMERO,
41     EPSILON
42 };
43
44 enum NoTerm {
45     strEmpty,
46     nt_programa, nt_declaraciones, nt_declaraciones_prima,
47     nt_declaracion, nt_tipo, nt_lista_var, nt_lista_var_prima,

```

```

43     nt_sentencias, nt_sentencias_prima, nt_sentencia,
44     nt_expression, nt_expression_prima,
45     nt_termino, nt_termino_prima, nt_factor
46 };
47
48 #endif // !__SYMBOLS_HPP__

```

9. Cargar $N' \cup \Sigma$ en *ParserLL.cpp*.
10. Cargar P' en *ParserLL.cpp*.
11. Cargar la tabla de análisis sintáctico predictivo en *ParserLL.cpp*.
12. Implementar el algoritmo de análisis sintáctico de descenso predictivo.

Se unificó la resolución de los ejercicios 9 al 12 adaptando el algoritmo a la plantilla provista de la clase *ParserLL*. Se utilizó un desfase (`offset = 100`) en las llaves del mapa para evitar colisiones numéricas entre Terminales y No-Terminales al registrarlos en la gramática.

El algoritmo predictivo utiliza una pila de objetos de tipo *Symbol* (`stack<Symbol>`). En cada iteración, se extrae el elemento tope (X) y se evalúa su tipo. Si es un terminal, se realiza la comparación directa con la entrada consumiéndola en caso de coincidencia. Si es un No-Terminal, se emplea un iterador (`accion`) para buscar de forma eficiente en la tabla predictiva la regla correspondiente, y se inserta el cuerpo de la producción a la pila en orden inverso.

A continuación se presenta el código íntegro implementado en *ParserLL.cpp*:

```

1 #include "headers/ParserLL.hpp"
2 #include <iostream>
3 #include <cstdlib>
4
5 // Usaremos un offset de 100 para diferenciar los ID de No-Terminales
6 // ya que ambos inician en 0 dentro de sus respectivos enum.
7 const int NT_OFFSET = 100;
8
9 ParserLL::ParserLL(Lexer *lexer) {
10     this->lexer = lexer;
11 }
12 ParserLL::~ParserLL() {}
13
14 void ParserLL::loadSyms() {
15     grammar.addSymbol(t_eof, Symbol(t_eof));
16     grammar.addSymbol(INT, Symbol(INT));
17     grammar.addSymbol(FLOAT, Symbol(FLOAT));
18     grammar.addSymbol(IF, Symbol(IF));
19     grammar.addSymbol(ELSE, Symbol(ELSE));
20     grammar.addSymbol(WHILE, Symbol(WHILE));
21     grammar.addSymbol(PYC, Symbol(PYC));
22     grammar.addSymbol(COMA, Symbol(COMA));
23     grammar.addSymbol(ASIG, Symbol(ASIG));
24     grammar.addSymbol(MAS, Symbol(MAS));

```

```

25     grammar.addSymbol(MENOS, Symbol(MENOS));
26     grammar.addSymbol(MUL, Symbol(MUL));
27     grammar.addSymbol(DIV, Symbol(DIV));
28     grammar.addSymbol(LPAR, Symbol(LPAR));
29     grammar.addSymbol(RPAR, Symbol(RPAR));
30     grammar.addSymbol(IDENTIFICADOR, Symbol(IDENTIFICADOR));
31     grammar.addSymbol(NUMERO, Symbol(NUMERO));
32     grammar.addSymbol(EPSILON, Symbol(EPSILON));
33
34     grammar.addSymbol(NT_OFFSET + nt_programa, Symbol(nt_programa));
35     grammar.addSymbol(NT_OFFSET + nt_declaraciones, Symbol(
nt_declaraciones));
36     grammar.addSymbol(NT_OFFSET + nt_declaraciones_prima, Symbol(
nt_declaraciones_prima));
37     grammar.addSymbol(NT_OFFSET + nt_declaracion, Symbol(
nt_declaracion));
38     grammar.addSymbol(NT_OFFSET + nt_tipo, Symbol(nt_tipo));
39     grammar.addSymbol(NT_OFFSET + nt_lista_var, Symbol(nt_lista_var));
40     grammar.addSymbol(NT_OFFSET + nt_lista_var_prima, Symbol(
nt_lista_var_prima));
41     grammar.addSymbol(NT_OFFSET + nt_sentencias, Symbol(nt_sentencias)
);
42     grammar.addSymbol(NT_OFFSET + nt_sentencias_prima, Symbol(
nt_sentencias_prima));
43     grammar.addSymbol(NT_OFFSET + nt_sentencia, Symbol(nt_sentencia));
44     grammar.addSymbol(NT_OFFSET + nt_expresion, Symbol(nt_expresion));
45     grammar.addSymbol(NT_OFFSET + nt_expresion_prima, Symbol(
nt_expresion_prima));
46     grammar.addSymbol(NT_OFFSET + nt_termino, Symbol(nt_termino));
47     grammar.addSymbol(NT_OFFSET + nt_termino_prima, Symbol(
nt_termino_prima));
48     grammar.addSymbol(NT_OFFSET + nt_factor, Symbol(nt_factor));
49
50     grammar.start = NT_OFFSET + nt_programa;
51 }
52
53 void ParserLL::loadProds() {
54     grammar.addProd(1, Production(NT_OFFSET + nt_programa, {NT_OFFSET
+ nt_declaraciones, NT_OFFSET + nt_sentencias}));
55     grammar.addProd(2, Production(NT_OFFSET + nt_declaraciones, {
NT_OFFSET + nt_declaracion, NT_OFFSET + nt_declaraciones_prima}));
56     grammar.addProd(3, Production(NT_OFFSET + nt_declaraciones_prima,
{NT_OFFSET + nt_declaracion, NT_OFFSET + nt_declaraciones_prima}));
57     grammar.addProd(4, Production(NT_OFFSET + nt_declaraciones_prima,
{EPSILON}));
58     grammar.addProd(5, Production(NT_OFFSET + nt_declaracion, {
NT_OFFSET + nt_tipo, NT_OFFSET + nt_lista_var, PYC}));
59     grammar.addProd(6, Production(NT_OFFSET + nt_tipo, {INT}));
60     grammar.addProd(7, Production(NT_OFFSET + nt_tipo, {FLOAT}));
61     grammar.addProd(8, Production(NT_OFFSET + nt_lista_var, {
IDENTIFICADOR, NT_OFFSET + nt_lista_var_prima}));
62     grammar.addProd(9, Production(NT_OFFSET + nt_lista_var_prima, {
COMA, IDENTIFICADOR, NT_OFFSET + nt_lista_var_prima}));
63     grammar.addProd(10, Production(NT_OFFSET + nt_lista_var_prima, {

```

```

64     EPSILON}));
65     grammar.addProd(11, Production(NT_OFFSET + nt_sentencias, {
        NT_OFFSET + nt_sentencia, NT_OFFSET + nt_sentencias_prima}));
66     grammar.addProd(12, Production(NT_OFFSET + nt_sentencias_prima, {
        NT_OFFSET + nt_sentencia, NT_OFFSET + nt_sentencias_prima}));
67     grammar.addProd(13, Production(NT_OFFSET + nt_sentencias_prima, {
        EPSILON}));
68     grammar.addProd(14, Production(NT_OFFSET + nt_sentencia, {
        IDENTIFICADOR, ASIG, NT_OFFSET + nt_expresion, PYC}));
69     grammar.addProd(15, Production(NT_OFFSET + nt_sentencia, {IF, LPAR
        , NT_OFFSET + nt_expresion, RPAR, NT_OFFSET + nt_sentencias, ELSE,
        NT_OFFSET + nt_sentencias}));
70     grammar.addProd(16, Production(NT_OFFSET + nt_sentencia, {WHILE,
        LPAR, NT_OFFSET + nt_expresion, RPAR, NT_OFFSET + nt_sentencias}));
71     grammar.addProd(17, Production(NT_OFFSET + nt_expresion, {
        NT_OFFSET + nt_termino, NT_OFFSET + nt_expresion_prima}));
72     grammar.addProd(18, Production(NT_OFFSET + nt_expresion_prima, {
        MAS, NT_OFFSET + nt_termino, NT_OFFSET + nt_expresion_prima}));
73     grammar.addProd(19, Production(NT_OFFSET + nt_expresion_prima, {
        MENOS, NT_OFFSET + nt_termino, NT_OFFSET + nt_expresion_prima}));
74     grammar.addProd(20, Production(NT_OFFSET + nt_expresion_prima, {
        EPSILON}));
75     grammar.addProd(21, Production(NT_OFFSET + nt_termino, {NT_OFFSET
        + nt_factor, NT_OFFSET + nt_termino_prima}));
76     grammar.addProd(22, Production(NT_OFFSET + nt_termino_prima, {MUL,
        NT_OFFSET + nt_factor, NT_OFFSET + nt_termino_prima}));
77     grammar.addProd(23, Production(NT_OFFSET + nt_termino_prima, {DIV,
        NT_OFFSET + nt_factor, NT_OFFSET + nt_termino_prima}));
78     grammar.addProd(24, Production(NT_OFFSET + nt_termino_prima, {
        EPSILON}));
79     grammar.addProd(25, Production(NT_OFFSET + nt_factor, {LPAR,
        NT_OFFSET + nt_expresion, RPAR}));
80     grammar.addProd(26, Production(NT_OFFSET + nt_factor, {
        IDENTIFICADOR}));
81     grammar.addProd(27, Production(NT_OFFSET + nt_factor, {NUMERO}));
82 }
83 void ParserLL::loadTable() {
84     table[nt_programa][INT] = 1; table[nt_programa][FLOAT] = 1;
85     table[nt_declaraciones][INT] = 2; table[nt_declaraciones][FLOAT] =
        2;
86     table[nt_declaraciones_prima][INT] = 3; table[
        nt_declaraciones_prima][FLOAT] = 3;
87     table[nt_declaraciones_prima][IDENTIFICADOR] = 4; table[
        nt_declaraciones_prima][IF] = 4; table[nt_declaraciones_prima][
        WHILE] = 4;
88     table[nt_declaracion][INT] = 5; table[nt_declaracion][FLOAT] = 5;
89     table[nt_tipo][INT] = 6; table[nt_tipo][FLOAT] = 7;
90     table[nt_lista_var][IDENTIFICADOR] = 8;
91     table[nt_lista_var_prima][COMA] = 9; table[nt_lista_var_prima][PYC
        ] = 10;
92     table[nt_sentencias][IDENTIFICADOR] = 11; table[nt_sentencias][IF]
        = 11; table[nt_sentencias][WHILE] = 11;
93     table[nt_sentencias_prima][IDENTIFICADOR] = 12; table[

```

```

nt_sentencias_prima][IF] = 12; table[nt_sentencias_prima][WHILE] =
12;
94   table[nt_sentencias_prima][ELSE] = 13; table[nt_sentencias_prima][
t_eof] = 13;
95   table[nt_sentencia][IDENTIFICADOR] = 14; table[nt_sentencia][IF] =
15; table[nt_sentencia][WHILE] = 16;
96   table[nt_expresion][IDENTIFICADOR] = 17; table[nt_expresion][
NUMERO] = 17; table[nt_expresion][LPAR] = 17;
97   table[nt_expresion_prima][MAS] = 18; table[nt_expresion_prima][
MENOS] = 19; table[nt_expresion_prima][PYC] = 20; table[
nt_expresion_prima][RPAR] = 20;
98   table[nt_termino][IDENTIFICADOR] = 21; table[nt_termino][NUMERO] =
21; table[nt_termino][LPAR] = 21;
99   table[nt_termino_prima][MUL] = 22; table[nt_termino_prima][DIV] =
23; table[nt_termino_prima][MAS] = 24;
100  table[nt_termino_prima][MENOS] = 24; table[nt_termino_prima][PYC]
= 24; table[nt_termino_prima][RPAR] = 24;
101  table[nt_factor][LPAR] = 25; table[nt_factor][IDENTIFICADOR] = 26;
table[nt_factor][NUMERO] = 27;
102 }
103
104 int ParserLL::parse() {
105     stack<Symbol> pila;
106     map<Token, int>::iterator accion;
107     vector<int> body;
108     Symbol X;
109
110     token = eat();
111     pila.push(grammar.getSym(t_eof));
112     pila.push(grammar.getSym(grammar.start));
113
114     while (!pila.empty()) {
115         X = pila.top();
116         pila.pop();
117
118         if (X.getType() == terminal) {
119             if (X.getToken() == EPSILON) continue;
120             else if (X.getToken() == token) {
121                 if (token == t_eof) break;
122                 token = eat();
123             } else {
124                 error("Error de sintaxis: Se esperaba otro token");
125                 return 1;
126             }
127         } else {
128             NoTerm nt = X.getNoTerm();
129             accion = table[nt].find(token);
130
131             if (accion != table[nt].end()) {
132                 int prodId = accion->second;
133                 Production p = grammar.getProd(prodId);
134                 body = p.getBody();
135
136                 for (int i = body.size() - 1; i >= 0; --i) {

```

```

137         pila.push(grammar.getSym(body[i]));
138     }
139     } else {
140         error("Error de sintaxis: Token inesperado");
141         return 1;
142     }
143 }
144 }
145
146 aceptar("!Análisis sintactico exitoso! La cadena es aceptada.");
147 return 0;
148 }
149
150 void ParserLL::error(string msg) {
151     cout << msg << " en la linea " << lexer->getLine() << endl;
152     exit(EXIT_FAILURE);
153 }
154
155 Token ParserLL::eat() { return (Token) lexer->yylex(); }
156
157 void ParserLL::aceptar(string msg) { cout << msg << endl; }

```

```

edwin-rodriguez@edwin-rodriguez-OMEN-by-HP-Laptop-15-dclxxx:~/Escritorio/ciencias/semestre 8/Compiladores/Laboratorio/Prácticas/P3/src$ ./comp prueba
¡Análisis sintáctico exitoso! La cadena es aceptada.
edwin-rodriguez@edwin-rodriguez-OMEN-by-HP-Laptop-15-dclxxx:~/Escritorio/ciencias/semestre 8/Compiladores/Laboratorio/Prácticas/P3/src$

```

Figura 1: Ejecución exitosa del analizador sintáctico predictivo LL(1) procesando el archivo de prueba.

Extras

13. Documentar el código.

Se añadió documentación detallada en el archivo `ParserLL.cpp`. Se explicaron las secciones de carga de la gramática, la inicialización de la tabla predictiva, la lógica del ciclo principal del parser, aclarando el uso de la pila de símbolos, el manejo de transiciones ϵ y el mecanismo de desfase de llaves.

14. Proponer 4 archivos de prueba nuevos, 2 válidos y 2 inválidos.

Archivos Válidos:

- `prueba_valida_1`: Con variables y operaciones que respetan la precedencia.

```

1 int a, b;
2 float c;
3 a = 5;
4 b = 10;
5 c = a + b * 2;

```

- `prueba_valida_2`: Con estructuras de control anidadas.

```

1 int x;
2 x = 1;
3 while (x)
4     if (x)
5         x = x + 1;
6     else
7         x = x - 1;

```

Archivos Inválidos:

- prueba_invalida_1: Error de sintaxis por no poner paréntesis en el ciclo `while`.

```

1 int i;
2 while i
3     i = i + 1;

```

- prueba_invalida_2: Intentar declarar una variable después de haber iniciado con las sentencias.

```

1 int a;
2 a = 10;
3 float b;

```

Comprensión de Conceptos: Análisis Predictivo LL(1)

A diferencia del análisis sintáctico de descenso recursivo de la práctica anterior, donde cada No-Terminal se implementa como una función consumiendo memoria en la pila de llamadas del sistema, el analizador predictivo LL(1) utiliza una tabla y utiliza una estructura de datos tipo `stack` explícita.

LL(1) significa que el analizador lee la entrada de Izquierda a derecha (*Left-to-right*), produce una derivación por la Izquierda (*Leftmost derivation*) y necesita mirar solo un token de anticipación para tomar decisiones deterministas gracias a los FIRST y FOLLOW.

Investigación y Resolución de Problemas

Durante la implementación del `ParserLL.cpp`, tuve un problema al utilizar la clase `Grammar`: los números de los símbolos Terminales (comenzando en 0) eran los mismos que los números de los No-Terminales (que también inician en 0).

Si no se resolvía, el mapa de símbolos habría sobrescrito las llaves, confundiendo al terminal INT con algún No-Terminal.

Solución: Investigué y usé un desfase (*offset*) de valor 100 para los No-Terminales (`NT_OFFSET = 100`) al momento de cargarlos en `loadSyms()` y al hacer las producciones en `loadProds()`. Esto hizo identificadores únicos en el parser.

Análisis de Resultados y Conclusiones

Los resultados al ejecutar confirman que funciona bien la tabla predictiva. Por lo tanto, eliminar la recursividad por la izquierda y la factorización son pasos muy importantes; si la gramática G' no hubiera eliminado esto, la intersección de los FIRST habría hecho celdas en la tabla M con más de una regla de producción, tirando por completo el determinismo del autómata.